

OnSite Technical Documentation

1. Architecture Overview

OnSite is built entirely on React Native. The core recognition and liveness logic runs on-device using ML Kit (for extraction) and ONNX Runtime (for evaluation). No network connection is required during the enrolment or verification phases.

2. Model Details

- SFace (face_recognition_sface_2021dec.onnx): A MobileFaceNet variant quantized to INT8. Produces 512-D L2-normalized embeddings for cosine matching. Size: ~37MB uncompressed.
- MiniFASNet (minifasnet_int8.onnx): Extremely lightweight passive liveness detector quantized to INT8. Size: ~1.7MB.
- Target App Size: With standard APK/IPA asset compression, the footprint fits neatly within the ~20MB target overhead requirement.

3. Liveness & Anti-Spoofing

A dual-layer approach is implemented in the livenessGate module:

- a) Passive: MiniFASNet evaluates texture to detect printed masks or screens.
- b) Active: Randomized challenges force the user to interact (blink, smile, turn head). This is evaluated using ML Kit FaceSignals (leftEyeOpenProbability, eulerY, eulerZ).

4. Sync & Purge Mechanism

Every verification creates a signed VerificationRecord. These records are queued locally in an encrypted store. When a network connection is detected, syncScheduler flushes pending records to an AWS endpoint. Once the server confirms receipt (HTTP 200), the local purge module deletes the record and associated templates to free device storage and maintain privacy.

5. Security

All templates are XOR encrypted using a 256-bit symmetric key securely stored in Android Keystore / iOS Keychain. Raw images are never saved to disk.